Universidad del Valle de Guatemala
Facultad de Ingeniería
POO
Segundo Ciclo, 2026



Laboratorio 2

Antonio Molina - 26878

1. Análisis

Requisitos funcionales

- El programa debe permitir crear un nuevo caso de investigación.
- Cada caso debe guardar el nombre del caso, código de identificación y nombre del detective responsable.
- Al crear un nuevo caso, este debe comenzar sin ubicaciones registradas y sin pistas encontradas.
- Cada caso puede tener un máximo de 5 ubicaciones.
- Las ubicaciones deben almacenarse en un arreglo básico de objetos de tamaño 5.
- Las posiciones del arreglo que no tengan una ubicación deben contener null.
- Cada ubicación debe guardar código, nombre, dirección o descripción, nivel de riesgo y estado.
- El nivel de riesgo debe encontrarse entre 1 y 10.
- Si se intenta crear una ubicación con un riesgo menor que 1 o mayor que 10, se debe generar una IllegalArgumentException.
- Para registrar una ubicación, el usuario debe indicar la posición donde desea almacenarla.
- La posición indicada debe encontrarse dentro de los límites del arreglo.
- La posición seleccionada debe estar vacía antes de registrar una ubicación.
- El sistema debe permitir consultar todas las ubicaciones registradas.
- Al consultar las ubicaciones no se deben mostrar como existentes las posiciones que contengan null.
- El sistema debe permitir consultar una ubicación utilizando su posición dentro del arreglo.
- Si la posición indicada no existe o contiene null, el programa debe informar sin finalizar inesperadamente.
- El sistema debe permitir modificar el nivel de riesgo de una ubicación registrada.
- El sistema debe permitir modificar el estado de una ubicación registrada.
- El nuevo nivel de riesgo también debe encontrarse entre 1 y 10.
- No se debe permitir modificar una posición que contenga null.
- El sistema debe permitir descartar una ubicación registrada.
- Al descartar una ubicación, su posición dentro del arreglo debe volver a contener null.
- El sistema debe permitir registrar una cantidad dinámica de pistas.
- Las pistas deben almacenarse obligatoriamente utilizando un ArrayList.
- Cada pista debe guardar código, descripción, tipo de evidencia, nivel de importancia y nivel de confiabilidad.
- El nivel de importancia debe encontrarse entre 1 y 10.
- El nivel de confiabilidad debe encontrarse entre 0 y 100.
- Si la importancia o confiabilidad no cumplen sus rangos, se debe generar una IllegalArgumentException.
- No se deben permitir dos pistas con el mismo código.
- El sistema debe permitir consultar todas las pistas registradas.
- Si el ArrayList está vacío, el programa debe manejar correctamente la situación.
- El sistema debe permitir buscar una pista mediante su código recorriendo el ArrayList.

- El sistema debe permitir modificar una pista registrada.
- Los nuevos valores de una pista deben cumplir las mismas validaciones utilizadas al crearla.
- El sistema debe permitir eliminar una pista del ArrayList.
- El sistema debe mostrar la cantidad de ubicaciones registradas.
- El sistema debe mostrar la cantidad de espacios disponibles para ubicaciones.
- El sistema debe encontrar la ubicación con mayor nivel de riesgo.
- El sistema debe mostrar la cantidad de pistas registradas.
- El sistema debe encontrar la pista con mayor nivel de importancia.
- El sistema debe encontrar la pista con mayor nivel de confiabilidad.
- El sistema debe calcular el promedio del nivel de importancia de las pistas registradas.
- Los cálculos relacionados con pistas deben utilizar solamente los objetos registrados en el ArrayList.
- No se deben realizar cálculos que necesiten pistas existentes cuando el ArrayList se encuentre vacío.
- Si el usuario escribe un tipo de dato incorrecto cuando se espera un número, se debe manejar una InputMismatchException.
- Después de una entrada incorrecta se debe limpiar el Scanner para que el programa pueda continuar.
- El programa debe utilizar bloques try-catch para manejar las excepciones.
- El programa debe utilizar por lo menos un bloque finally.
- Después de manejar una excepción, el programa debe continuar funcionando.
- El programa debe mostrar un menú con las 13 opciones indicadas en el enunciado.

Propiedades, métodos de las clases

Clase Ubicacion

La clase Ubicacion representa uno de los lugares que deben investigarse durante el caso. Guarda la información propia del lugar y permite modificar su nivel de riesgo y estado.

Propiedades

- codigo
- nombre
- direccion
- nivelRiesgo
- estado

Métodos

- Constructor
- getCodigo()
- getNombre()
- getDireccion()
- getNivelRiesgo()
- getEstado()
- setNivelRiesgo()
- setEstado()
-

Clase Pista

La clase Pista representa una evidencia encontrada durante la investigación. Guarda sus datos y permite modificar la información que puede cambiar durante el caso.

Propiedades

- codigo
- descripcion
- tipoEvidencia
- nivelImportancia
- nivelConfiabilidad

Métodos

- Constructor
- getCodigo()
- getDescripcion()
- getTipoEvidencia()
- getNivelImportancia()
- getNivelConfiabilidad()
- setDescripcion()
- setTipoEvidencia()
- setNivelImportancia()
- setNivelConfiabilidad()

Clase Caso

La clase Caso representa la investigación que está realizando la agencia.

Guarda los datos generales del caso, las ubicaciones y las pistas. También se encarga de las operaciones relacionadas con las dos estructuras, como registrar, consultar, modificar, eliminar y realizar los cálculos necesarios.

Propiedades

- nombreCaso
- codigoIdentificacion
- detectiveResponsable
- ubicaciones
- pistas

Métodos

- Constructor
- getNombreCaso()
- getCodigoIdentificacion()
- getDetectiveResponsable()
- getUbicaciones()
- getPistas()
- registrarUbicacion()
- obtenerUbicacion()
- modificarUbicacion()
- descartarUbicacion()
- registrarPista()
- buscarPista()

- eliminarPista()
- cantidadUbicaciones()
- espaciosDisponibles()
- ubicacionMayorRiesgo()
- cantidadPistas()
- pistaMayorImportancia()
- pistaMayorConfiabilidad()
- promedioImportancia()

Clase Main

La clase Main se encarga de crear el caso, mostrar el menú, solicitar datos, mostrar resultados y manejar las excepciones que puedan ocurrir durante la interacción con el usuario.

Propiedades

- teclado
- caso

Métodos

- main()
- nuevoCaso()
- registrarUbicacion()
- consultarUbicaciones()
- consultarUbicacion()
- modificarUbicacion()
- descartarUbicacion()
- registrarPista()
- consultarPistas()
- buscarPista()
- modificarPista()
- eliminarPista()
- mostrarReporte()

Tipo de las propiedades y métodos

Ubicacion

codigo (String): guarda el código que identifica la ubicación.
nombre (String): guarda el nombre de la ubicación.
direccion (String): guarda la dirección o descripción del lugar.
nivelRiesgo (int): guarda el riesgo de la ubicación. Debe encontrarse entre 1 y 10.
estado (String): guarda el estado actual de la ubicación.
Ubicacion(String codigo, String nombre, String direccion, int nivelRiesgo, String estado): constructor que recibe los datos necesarios para crear una ubicación. Debe validar el nivel de riesgo.
getCodigo() : String: devuelve el código.
getNombre() : String: devuelve el nombre.
getDireccion() : String: devuelve la dirección.
getNivelRiesgo() : int: devuelve el nivel de riesgo.
getEstado() : String: devuelve el estado.

setNivelRiesgo(int nivelRiesgo) : void: recibe el nuevo nivel de riesgo. Si no está entre 1 y 10 debe generar una IllegalArgumentException.
setEstado(String estado) : void: recibe y modifica el estado.

Pista

codigo (String): guarda el código que identifica la pista.
descripcion (String): guarda la descripción de la evidencia.
tipoEvidencia (String): guarda el tipo de evidencia.
nivelImportancia (int): guarda el nivel de importancia entre 1 y 10.
nivelConfiabilidad (int): guarda el nivel de confiabilidad entre 0 y 100.
Pista(String codigo, String descripcion, String tipoEvidencia, int nivelImportancia, int nivelConfiabilidad): crea una pista y valida importancia y confiabilidad.
getCodigo() : String: devuelve el código.
getDescripcion() : String: devuelve la descripción.
getTipoEvidencia() : String: devuelve el tipo de evidencia.
getNivelImportancia() : int: devuelve la importancia.
getNivelConfiabilidad() : int: devuelve la confiabilidad.
setDescripcion(String descripcion) : void: modifica la descripción.
setTipoEvidencia(String tipoEvidencia) : void: modifica el tipo de evidencia.
setNivelImportancia(int nivelImportancia) : void: modifica la importancia y genera una IllegalArgumentException si no está entre 1 y 10.
setNivelConfiabilidad(int nivelConfiabilidad) : void: modifica la confiabilidad y genera una IllegalArgumentException si no está entre 0 y 100.

Caso

nombreCaso (String): guarda el nombre del caso.
codigoIdentificacion (String): guarda el código que identifica la investigación.
detectiveResponsable (String): guarda el nombre del detective responsable.
ubicaciones (Ubicacion[]): arreglo básico que almacena como máximo 5 objetos Ubicacion. Las posiciones sin utilizar contienen null.
pistas (ArrayList<Pista>): colección dinámica que almacena los objetos Pista encontrados durante la investigación.

Métodos principales

Caso(String nombreCaso, String codigoIdentificacion, String detectiveResponsable): crea el caso, crea el arreglo de 5 ubicaciones e inicializa el ArrayList vacío.
registrarUbicacion(int posicion, Ubicacion ubicacion) : boolean: recibe una posición y una ubicación. La almacena solamente si la posición existe y contiene null.
obtenerUbicacion(int posicion) : Ubicacion: devuelve la ubicación almacenada en la posición indicada.
modificarUbicacion(int posicion, int riesgo, String estado) : boolean: modifica una ubicación existente.
descartarUbicacion(int posicion) : boolean: coloca null en la posición correspondiente si existe una ubicación.
registrarPista(Pista pista) : boolean: agrega la pista al ArrayList si su código no está repetido.
buscarPista(String codigo) : Pista: recorre el ArrayList y devuelve la pista con el código solicitado o null si no existe.

eliminarPista(String codigo) : boolean: busca una pista y la elimina del ArrayList.
cantidadUbicaciones() : int: cuenta las posiciones del arreglo que no contienen null.
espaciosDisponibles() : int: cuenta las posiciones que contienen null.
ubicacionMayorRiesgo() : Ubicacion: devuelve la ubicación registrada con el nivel de riesgo más alto.
cantidadPistas() : int: devuelve la cantidad de objetos almacenados en el ArrayList.
pistaMayorImportancia() : Pista: devuelve la pista con mayor importancia.
pistaMayorConfiabilidad() : Pista: devuelve la pista con mayor confiabilidad.
promedioImportancia() : double: calcula el promedio utilizando las pistas registradas.

¿Qué propiedad utiliza un arreglo básico?

La propiedad Ubicacion[] ubicaciones de la clase Caso. El arreglo almacenará objetos Ubicacion y tendrá un tamaño fijo de: new Ubicacion[5]
Las posiciones que todavía no tengan una ubicación registrada contendrán null.
Por ejemplo:
Posición 0 - Ubicacion
Posición 1 - null
Posición 2 - Ubicacion
Posición 3 - null
Posición 4 - null

¿Qué propiedad utiliza ArrayList?

La propiedad: ArrayList<Pista> pistas de la clase Caso. Almacenará objetos Pista y no tendrá un límite fijo de elementos. Se inicializará: pistas = new ArrayList<Pista>(). No utilizaremos: List<Pista>

Modificadores de visibilidad

- En Ubicacion, todos los atributos serán private y el constructor, getters y setters serán public.
- En Pista, todos los atributos serán private y sus métodos serán public.
- En Caso, sus datos generales, el arreglo y el ArrayList serán private. El constructor y los métodos necesarios para administrar la investigación serán public.
- En Main, teclado y caso serán private static. Los métodos utilizados para controlar las opciones del menú serán public static.

Parámetros requeridos

Ubicacion

- El constructor recibe código, nombre, dirección, nivel de riesgo y estado.
- setNivelRiesgo() recibe el nuevo nivel.
- setEstado() recibe el nuevo estado.
- Los getters no reciben parámetros.

Pista

- El constructor recibe código, descripción, tipo de evidencia, nivel de importancia y nivel de confiabilidad.
- Cada setter recibe el nuevo valor correspondiente.
- Los getters no reciben parámetros.

Caso

- registrarUbicacion() recibe posición y objeto Ubicacion.
- obtenerUbicacion() recibe posición.
- modificarUbicacion() recibe posición, nuevo riesgo y nuevo estado.
- descartarUbicacion() recibe posición.
- registrarPista() recibe un objeto Pista.
- buscarPista() recibe código.
- eliminarPista() recibe código.

Valores iniciales y validaciones

Cuando se crea un nuevo Caso, se reciben nombre, código y detective responsable. Se crea: ubicaciones = new Ubicacion[5]
Las cinco posiciones comienzan automáticamente con null.
También se crea: pistas = new ArrayList<Pista>(); por lo que inicialmente no existen pistas.
Antes de crear o modificar una Ubicacion, el nivel de riesgo debe estar entre 1 y 10. Si no se cumple, se genera una IllegalArgumentException. Antes de crear o modificar una Pista, la importancia debe estar entre 1 y 10 y la confiabilidad entre 0 y 100. Si alguno es inválido, también se genera una IllegalArgumentException.

¿Cómo determinar si una posición contiene una ubicación o null?

Se utilizará el índice del arreglo.
Por ejemplo:

```
if (ubicaciones[posicion] == null) {
   //La posicion esta disponible
}
```

Si es diferente de null:

```
if (ubicaciones[posicion] != null) {
   //Existe una ubicacion registrada
}
```

También se debe comprobar primero que:
posicion >= 0 && posicion < ubicaciones.length
para evitar acceder a una posición que no existe.

Búsqueda, modificación y eliminación en el ArrayList

Para buscar una pista se recorrerá el ArrayList utilizando un índice. Se comparará el código de cada pista con el código buscado. Cuando se encuentre la coincidencia se devolverá el objeto. Para modificar una pista, primero se utilizará la búsqueda y después se cambiarán sus datos mediante los setters. Los nuevos valores de importancia y confiabilidad volverán a ser validados. Para eliminar una pista, primero se localizará el objeto y después se utilizará: pistas.remove(pista)

Excepciones, try-catch y finally

InputMismatchException

Puede ocurrir cuando Scanner espera un número y el usuario escribe texto. Por ejemplo:
Ingrese el nivel de riesgo: hola
El programa utilizará try-catch para capturar la InputMismatchException, mostrar un mensaje y limpiar la entrada incorrecta con teclado.nextLine(). Después deberá regresar al menú y continuar funcionando.

IllegalArgumentException

Será generada cuando los valores de los objetos no cumplan las reglas.
En Ubicacion: nivelRiesgo < 1 o nivelRiesgo > 10
En Pista: nivelImportancia < 1 o nivelImportancia > 10 o: nivelConfiabilidad < 0 o nivelConfiabilidad > 100
Los métodos de Main que creen o modifiquen estos objetos utilizarán try-catch para manejar estas excepciones.

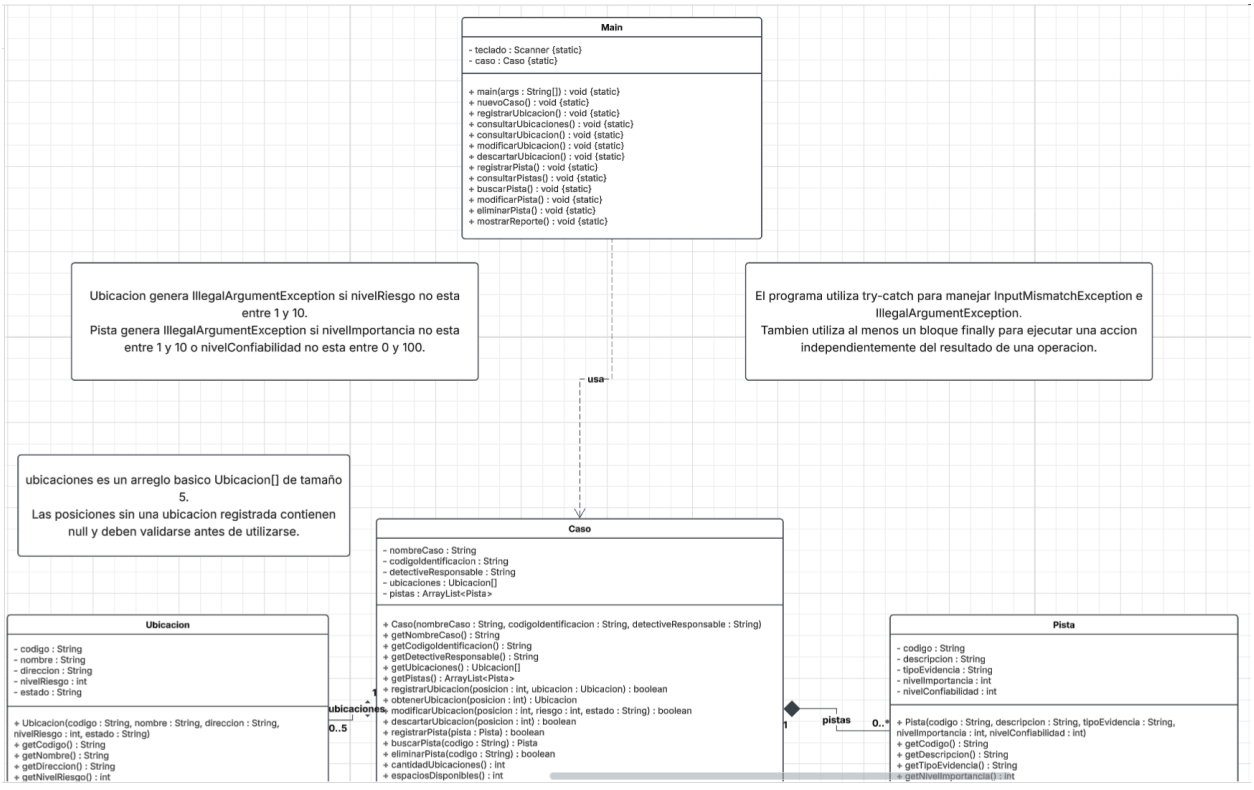
Finally

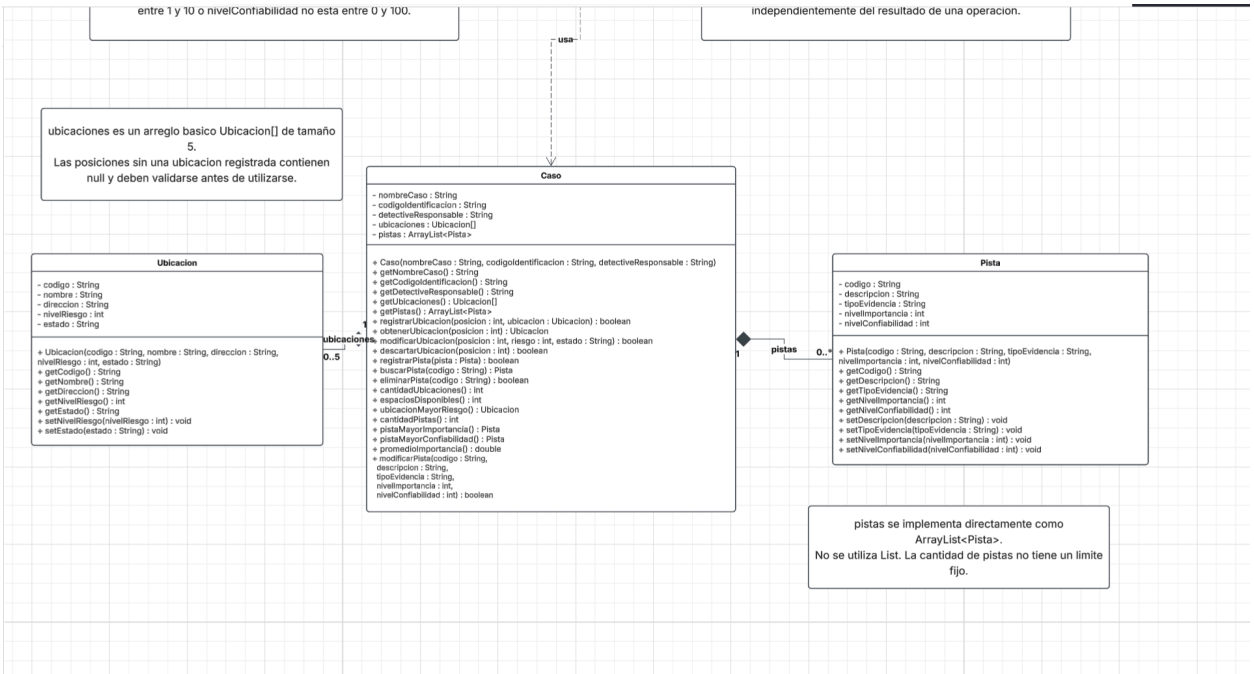
Utilizaremos por lo menos un finally en una operación como el registro de una ubicación.
Por ejemplo: finally {

```
    System.out.println("Proceso de registro de ubicacion finalizado.");
}
```

Este mensaje aparecerá tanto si la ubicación se registró correctamente como si ocurrió una excepción.

UML


Main
- teclado : Scanner {static}
- caso : Caso {static}
+ main(args : String[]) : void {static}
+ nuevoCaso() : void {static}
+ registrarUbicacion() : void {static}
+ consultarUbicaciones() : void {static}
+ consultarUbicacion() : void {static}
+ modificarUbicacion() : void {static}
+ descartarUbicacion() : void {static}
+ registrarPista() : void {static}
+ consultarPistas() : void {static}
+ buscarPista() : void {static}
+ modificarPista() : void {static}
+ eliminarPista() : void {static}
+ mostrarReporte() : void {static}
Ubicacion genera IllegalArgumentException si nivelRiesgo no esta entre 1 y 10.
Pista genera IllegalArgumentException si nivelImportancia no esta entre 1 y 10 o nivelConfiabilidad no esta entre 0 y 100.
El programa utiliza try-catch para manejar InputMismatchException e IllegalArgumentException.
Tambien utiliza al menos un bloque finally para ejecutar una accion independientemente del resultado de una operacion.
usa
ubicaciones es un arreglo basico Ubicacion[] de tamaño 5.
Las posiciones sin una ubicacion registrada contienen null y deben validarse antes de utilizarse.
Caso
- nombreCaso : String
- codigoIdentificacion : String
- detectiveResponsable : String
- ubicaciones : Ubicacion[]
- pistas : ArrayList<Pista>
+ Caso(nombreCaso : String, codigoIdentificacion : String, detectiveResponsable : String)
+ getNombreCaso() : String
+ getCodigoIdentificacion() : String
+ getDetectiveResponsable() : String
+ getUbicaciones() : Ubicacion[]
+ getPistas() : ArrayList<Pista>
+ registrarUbicacion(posicion : int, ubicacion : Ubicacion) : boolean
+ obtenerUbicacion(posicion : int) : Ubicacion
+ modificarUbicacion(posicion : int, riesgo : int, estado : String) : boolean
+ descartarUbicacion(posicion : int) : boolean
+ registrarPista(pista : Pista) : boolean
+ buscarPista(codigo : String) : Pista
+ eliminarPista(codigo : String) : boolean
+ cantidadUbicaciones() : int
+ espaciosDisponibles() : int
Ubicacion
- codigo : String
- nombre : String
- direccion : String
- nivelRiesgo : int
- estado : String
+ Ubicacion(codigo : String, nombre : String, direccion : String, nivelRiesgo : int, estado : String)
+ getCodigo() : String
+ getNombre() : String
+ getDireccion() : String
+ getNivelRiesgo() : int
Pista
- codigo : String
- descripcion : String
- tipoEvidencia : String
- nivelImportancia : int
- nivelConfiabilidad : int
+ Pista(codigo : String, descripcion : String, tipoEvidencia : String, nivelImportancia : int, nivelConfiabilidad : int)
+ getCodigo() : String
+ getDescripcion() : String
+ getTipoEvidencia() : String
+ getNivelImportancia() : int
ubicaciones
1
0..5
pistas
1
0..*


entre 1 y 10 o nivelConfiabilidad no esta entre 0 y 100.
independientemente del resultado de una operacion.
usa
ubicaciones es un arreglo basico Ubicacion[] de tamaño 5.
Las posiciones sin una ubicacion registrada contienen null y deben validarse antes de utilizarse.
Caso
- nombreCaso : String
- codigoIdentificacion : String
- detectiveResponsable : String
- ubicaciones : Ubicacion[]
- pistas : ArrayList<Pista>
+ Caso(nombreCaso : String, codigoIdentificacion : String, detectiveResponsable : String)
+ getNombreCaso() : String
+ getCodigoIdentificacion() : String
+ getDetectiveResponsable() : String
+ getUbicaciones() : Ubicacion[]
+ getPistas() : ArrayList<Pista>
+ registrarUbicacion(posicion : int, ubicacion : Ubicacion) : boolean
+ obtenerUbicacion(posicion : int) : Ubicacion
+ modificarUbicacion(posicion : int, riesgo : int, estado : String) : boolean
+ descartarUbicacion(posicion : int) : boolean
+ registrarPista(pista : Pista) : boolean
+ buscarPista(codigo : String) : Pista
+ eliminarPista(codigo : String) : boolean
+ cantidadUbicaciones() : int
+ espaciosDisponibles() : int
+ ubicacionMayorRiesgo() : Ubicacion
+ cantidadPistas() : int
+ pistaMayorImportancia() : Pista
+ pistaMayorConfiabilidad() : Pista
+ promedioImportancia() : double
+ modificarPista(codigo : String, descripcion : String, tipoEvidencia : String, nivelImportancia : int, nivelConfiabilidad : int) : boolean
Ubicacion
- codigo : String
- nombre : String
- direccion : String
- nivelRiesgo : int
- estado : String
+ Ubicacion(codigo : String, nombre : String, direccion : String, nivelRiesgo : int, estado : String)
+ getCodigo() : String
+ getNombre() : String
+ getDireccion() : String
+ getNivelRiesgo() : int
+ getEstado() : String
+ setNivelRiesgo(nivelRiesgo : int) : void
+ setEstado(estado : String) : void
Pista
- codigo : String
- descripcion : String
- tipoEvidencia : String
- nivelImportancia : int
- nivelConfiabilidad : int
+ Pista(codigo : String, descripcion : String, tipoEvidencia : String, nivelImportancia : int, nivelConfiabilidad : int)
+ getCodigo() : String
+ getDescripcion() : String
+ getTipoEvidencia() : String
+ getNivelImportancia() : int
+ getNivelConfiabilidad() : int
+ setDescripcion(descripcion : String) : void
+ setTipoEvidencia(tipoEvidencia : String) : void
+ setNivelImportancia(nivelImportancia : int) : void
+ setNivelConfiabilidad(nivelConfiabilidad : int) : void
ubicaciones
1
0..5
pistas
1
0..*
pistas se implementa directamente como ArrayList<Pista>.
No se utiliza List. La cantidad de pistas no tiene un limite fijo.